

SafeTransition: exact rational certification of transition safety for sustainability assessment

Amin Abaee^a

^a*Independent Researcher, ORCID: 0000-0002-0019-1842, amin_abaee@ut.ac.ir*

Abstract

SafeTransition, a Python library for certifying the transition safety of sustainability assessments, is presented. The library implements the assessment-operator framework: typed floor constraints evaluated on supplied exact or certified conservative reachability tubes, scalarized aggregate operators at arbitrary positive weights, and backward recursions over state and belief spaces. Infeasibility is returned as a Farkas certificate, a nonnegative multiplier vector with vanishing weighted constraint sum and negative weighted bound; certificates serialize to JSON for an independent checker sharing no code with the library. All computations run in exact rational arithmetic with no third-party runtime dependencies; floating point appears only in rendered graphics. Twenty-four exact checks re-derive the companion deposit's benchmark, and a scaling study over five parameterized families verifies closed-form answers up to 256-variable eliminations and margins with 321-bit denominators. A single-file dashboard renders the readings with embedded verification provenance. The library is distributed under the MIT licence.

Keywords: sustainability assessment, viability theory, transition safety, Farkas certificate, exact arithmetic, composite indicators, multi-criteria decision analysis

1. Introduction

Composite indicators occupy a central place in sustainability monitoring. Aggregates reduce heterogeneous floor constraints on income, biomass, and social outcomes to a single weighted reading, and that reading is what dashboards, statutory reviews, and public communications display. The practice rests on a long modelling tradition, and its governance follows established frameworks for good model development and evaluation practice [12]. It also rests on a mathematical step that is easy to audit and easy to overlook: a weighted sum is certified positive at every audited snapshot, and the system behind the snapshots is declared safe.

The companion manuscripts show that this step can fail, and characterize exactly when it fails. In the assessment-operator framework of Abaee [1], a noncompensatory typed operator — which requires every floor to hold on the full trajectory, not at endpoints alone — can reject every plan that a scalarized aggregate operator licenses at a given weight; the separation is exhibited on an explicit rational witness datum and propagated through backward recursions. A resource-transition benchmark realizes the witness as a regulated fishery transition and verifies every quoted number in exact rational arithmetic. Under incomplete observation, the obstruction calculus of Abaee [2] complements viability assessment from the other side: it certifies that no observation-based policy is viable, through finitely checkable certificates — the common-action obstruction with an explicit Farkas multiplier vector, the observation-fibre criterion, and a finite-horizon belief-space recursion

that is sound and complete in finite systems. These results are mathematical; they describe what should be computed. They do not yet describe a tool that computes it.

This is the gap SafeTransition addresses. Reading a composite dashboard today, an analyst can recompute the aggregate with ordinary floating-point tooling, but has no standard way to re-derive the safety verdict behind it, to check that an infeasible plan menu is infeasible by certificate rather than by solver exit code, or to see which weight ratios license which plans. Numerical viability and control toolboxes compute kernels and invariant sets to high precision; multi-criteria decision analysis packages aggregate and rank; formal verification systems and satisfiability-modulo-theories solvers certify finite-state models and general constraint systems against their own specification semantics. What is missing in the intersection is a small, auditable layer in which the verdicts themselves — admissibility of a plan at a state, licensing at a weight, emptiness of a menu, emptiness of an epistemic kernel — are returned as exact rational objects that a third party can re-verify by hand or by machine, with the assessment semantics (floors, weights, licensed plan sets, partially observed beliefs) carried by the verdicts themselves rather than re-encoded.

This paper presents SafeTransition, a Python library that provides this layer. The related-software positioning of the paper was compiled against a preserved record of twelve verbatim web queries with dated result records (q1–q12, 2026), archived in the verification deposit; Section 5 reports what the searches covered and found. The library implements the five assessment operators and their accepted-state sets, the typed finite-graph backward recursion, the belief-space robust epistemic recursion, and the finitely checkable certificate family: exact rational Fourier–Motzkin elimination with provenance tracking, returning Farkas certificates and infeasibility margins; the finite common-action obstruction with a minimal conflicting subfamily; and the observation-fibre criterion. Every computation runs in exact rational arithmetic; floats appear only in rendered graphics. The library re-derives, in twenty-four exact checks, the resource-transition benchmark of the verification deposit, and renders the resulting readings as a single-file dashboard. It depends on nothing beyond the Python standard library, which makes the exactness claim structural rather than procedural: there is no numerical backend whose rounds of approximation would need to be audited.

The paper is a software description. Section 2 states the mathematical basis as implemented, with proofs delegated to the companion manuscripts. Section 3 describes the software architecture and the exact-arithmetic discipline. Section 4 illustrates the library on the benchmark, on the dashboard readings, and on the partial-observation demonstrations. Section 5 positions the library relative to existing software, and Section 6 records availability and requirements. Section 7 concludes.

2. Mathematical basis

2.1. Transition data and assessment operators

A typed transition datum comprises a phase state $z = (q, x, s_1, \dots, s_n)$ with a phase flag q , fund coordinates x , and typed floors s_i normalized to zero; a finite action set; a set of disturbances with, for each action and disturbance, an exact tube $\text{Tube}(a, d)$ — the set of states visited over the review period, given here as piecewise-linear coordinate paths on breakpoints at which every visited extreme is attained; a transition-safe set S ; and a destination set G reached under an endpoint reset. Actions in the library declare offsets on the breakpoints, together with an action-indexed worst-case disturbance, so that each tube is the exact visited set and no outer approximation enters.

Five operators are distinguished, following Abaee [1]. For a state z , the noncompensatory typed operator admits action a when every floor holds on $\text{Tube}(a, d)$ for every disturbance d and the successor lies in G ; the scalarized aggregate operator replaces the coordinate-wise condition by $w \cdot s \geq 0$ at weight w in the positive weight cone; the exact-tube physical and endpoint-only physical operators weaken the constraint set; and the typed-endpoint operator evaluates floors at endpoints only. Because endpoints are contained in tubes and typed sets in physical sets, the chain

$$E_{\text{typ}}(z) \subseteq E_w(z) \subseteq E_{\text{tube,phys}}(z) \subseteq E_{\text{end}}(z)$$

holds for every weight in the positive weight cone and for menus whose plans share a common disturbance set — the standing scope conditions of the operator framework — and the library verifies it exactly at any state on request (Section 3.3). The fifth operator continues the pattern on the typed side: $E_{\text{typ}}(z) \subseteq E_{\text{end,typ}}(z) \subseteq E_{\text{end}}(z)$, since typed floors constrain more than endpoints and more than physics alone. The accepted-state set $\mathcal{V}[E] = \{z : E(z) \neq \emptyset\}$ and the compensatory accepted set $\mathcal{V}_{\text{weak}} = \bigcap_w \mathcal{V}_w$ are derived from the operators; the witnessed separation $\mathcal{V}_{\text{typ}} \subseteq \mathcal{V}_{\text{weak}}$, strict in general, is what a composite dashboard cannot see (Section 4.1).

2.2. Recursions

Two backward recursions are implemented. The typed recursion propagates viability backwards through an explicit state–action graph: a state is k -step viable when some action keeps every visited state inside the safe set and lands in the $(k - 1)$ -step viable set. On explicit graphs with constant-time predicates the recursion is polynomial in graph size and horizon. The belief-space recursion addresses partially observed systems, following the robust epistemic construction of Abaee [2]: states carry observation labels, an action is admissible for a belief (a set of states consistent with the observations so far) when no state in its observation fibre enters a violation en route, and the belief updates to the set of labels reachable under the action. The recursion returns the sets $\mathcal{W}_k$ of k -step viable beliefs. Memberships certify viability in all finite systems; completeness — a belief outside $\mathcal{W}_N$ admits no viable observation-based policy over the horizon — holds when the observation is injective on reachable states, the exactness condition stated with the interface. A failure explainer accompanies a non-viable belief with its per-action counterexample: the exact reason each admissible-in-isolation action fails — a fibre state entering a violation en route, or a post-belief that is itself not viable at the previous level. Post-observation recourse failures — beliefs in $\mathcal{W}_1 \setminus \mathcal{W}_2$ — are the signature output (Section 4.3).

2.3. Certificates

Infeasibility is certified, not merely reported. For a stacked rational system $Au \leq b$, the library runs Fourier–Motzkin elimination with provenance tracking: every derived row carries its multiplier vector over the original rows, and when a contradiction $0 \leq c$, $c < 0$ is derived, the derived multipliers $\lambda \geq 0$ with $\lambda^\top A = 0$ and $\lambda^\top b < 0$ are returned as a Farkas certificate, normalized to sum to one (the certificate itself is invariant to positive scaling; the infeasibility margin $-\lambda^\top b > 0$ is defined by this normalization). The certificate is re-verified exactly before being returned. The finite common-action obstruction fires when the safe-action sets of pairwise-compatible states intersect emptily while each is nonempty — every state is individually viable under full information, yet no observation-based policy is viable; the library reports a minimal conflicting subfamily. The observation-fibre criterion decides whether an exact observation-only certifier can exist at all: one exists if and only if safe-set membership is constant on observation fibres, and a violating fibre is reported otherwise.

Module	Contents
<code>rational</code>	exact rational helpers (interpretation, rendering)
<code>datum</code>	typed transition data; the witness datum instance
<code>operators</code>	five operators, accepted-state sets, chain check
<code>recursion</code>	typed finite-graph recursion; belief-space recursion
<code>certificates</code>	Fourier–Motzkin with provenance; Farkas verification; common-action obstruction; fibre criterion
<code>indicators</code>	licensing thresholds, rescue threshold, blindness alarm
<code>dashboard</code>	single-file HTML rendering (inline SVG/CSS)
<code>benchmark</code>	twenty-four exact checks; verified schedule values; tube and benchmark certificates
<code>benchmarks</code>	scaling study: five families, answers known by construction (committed results)
<code>cli</code>	<code>verify</code> / <code>report</code> / <code>demo</code> / <code>certify</code>
<code>check_safe_transition_cert.py</code>	independent stdlib-only verifier for serialized certificates (repository root)

Table 1: Module organization of SafeTransition 1.2.1 (1 633 source lines, 621 test lines, plus the 250-line independent checker and the 291-line scaling study).

3. Software description

3.1. Architecture

The library is organized as nine modules plus the repository-root independent checker script (Table 1). The `rational` module provides the exact rational helpers on which every other module rests, and the `datum` module defines the transition-data interface and ships one fully specified instance, the witness datum of the companion manuscript, with destination gain $e = (1/4, 1/4)$ and rescue cost $c = 1$. Custom data subclass the interface. The `operators`, `recursion`, and `certificates` modules implement Section 2; the `indicators` and `dashboard` modules produce the readings of Section 4.2; the `benchmark` module carries the twenty-four exact checks of Section 4.1. A command-line interface exposes `verify`, `report`, `demo`, and `certify` — the last emits the serializable certificates of Section 4.3 for the independent checker.

3.2. The exact-arithmetic discipline

Every assessment, recursion, and certificate is computed in exact rational arithmetic. Rationals enter either as integers, as decimal strings, or as explicit numerator–denominator pairs; no implicit conversion from binary floating point exists anywhere in the library, so an input’s exactness is structural, not a matter of reviewer discipline. Floats occur in exactly two places, both rendering: SVG geometry in the dashboard and matplotlib coordinates in figure scripts. Checks never read floats. The distinction is the library’s design-centre claim: outputs are verifiable, not merely computed. A Farkas certificate, a licensing threshold, or a benchmark check can be re-verified by any rational-arithmetic implementation — including pencil and paper — and the test suite exercises exactly this: every certificate returned by the eliminator is re-verified against its defining identities before being returned, and the benchmark checks re-derive each deposited value symbolically (for example $\sigma(16/5) = 1088/125$ for the sustained-yield quota of the Schaefer realization). The discipline is not merely hygienic: the suite includes an adversarial instance on which exact and floating-point verdicts disagree. The trough condition $49 \cdot (1/49) - 1 = 0$ holds exactly and licenses

the plan; its binary floating-point evaluation yields -1.11×10^{-16} and would unlicense it. The library rejects floating-point inputs structurally, so the exact verdict is the only one it can return. Exactness also scales gracefully where floating point degrades: on a dyadic chain whose planted infeasibility gap is 2^{-t} , the certified margin keeps $t + 1$ denominator bits at $t = 320$, long after binary floating point has rounded the gap away (Section 3.4).

3.3. Interfaces and complexity

The application programming interface mirrors the mathematics: `E(datum, z, mode, w)` returns the action set of an operator at a state; `check_chain` verifies the operator chain exactly; `belief_backward` returns the belief-viability sets $\mathcal{W}_k$ and `explain_belief_failure` the per-action counterexample for a belief outside them; `certify_polyhedron` returns a feasibility verdict with its certificate; `weight_partition` returns the complete licensing arrangement along the weight ratio as a serializable certificate; `compute` returns the dashboard readings at a state, weight, and read plan. The typed recursion on explicit graphs is polynomial in graph size and horizon; the Fourier–Motzkin eliminator is exponential in the worst case, as exact elimination inherently is, which is why the certificate family targets the small stacked systems that transition menus produce (a handful of constraints; the witness instances eliminate in milliseconds). Belief enumeration is bounded — the bound is a parameter (default 4096 reachable beliefs) whose exhaustion raises an exception carrying no partial results; the bound affects exploration completeness, never the soundness of returned memberships.

The precision–performance trade-off is measured, not asserted. A scaling study sweeps five parameterized instance families whose answers are known by construction and asserted at every size: stacked Fourier–Motzkin chains with a planted infeasibility gap (chain length up to $k = 256$ variables), dyadic coefficient growth (exact margins certified at denominators 2^t up to $t = 320$), typed recursions on layered graphs (up to 21 000 states at horizon 20), cloned action menus (up to 512 actions through all five operators), and belief systems whose reachable belief count is exactly 2^m (the default enumeration bound of 4096 is reached exactly at $m = 12$ and exceeded at $m = 13$, where the call raises with no partial results). The study records wall-clock runtime, peak traced memory, eliminator row statistics (rows generated, peak working rows), certificate sizes and maximal numerator/denominator bit lengths, and independent-checker runtime (Table 2). Growth is unremarkable in the right sense: elimination is quadratic in the planted instances (the k -variable chain generates k rows and terminates at the contradiction; the certificate JSON reaches 335 kB at $k = 256$, with 13-bit coefficients, and the independent checker re-verifies it in 0.29 s); the typed recursion is linear in the state count (21 000 states in 2.1 s); belief enumeration is exponential in m , as the reachable set itself is, and the bound fires where designed. Coefficient growth inflates certificate size, not verdict times; the exact margin’s closed form $(1/5 - (k - 1)/4096)/(k + 1)$ is verified at every k .

3.4. Scaling of the exact pipeline

4. Illustrative examples

4.1. The resource-transition benchmark

The benchmark realizes the witness datum as a regulated fishery transition: stock biomass with Schaefer surplus $\sigma(B) = rB(1 - B/K)$ ($r = 4$, $K = 10$, $B_{\text{lim}} = 2$), a pulse-plus-closed-season quota plan, a mid-season heatwave strike, and a staged rebuild financed by a fund buy-back. Running `safetransition verify` executes twenty-four exact checks that re-derive the deposited

Family (largest instance)	Known answer	Largest-instance measurement
FM chain, $k = 256$	margin $(1/5 - 255/4096)/257$	1.59 s eliminator; 256 rows; 335 kB certificate; checker 0.29 s
Dyadic growth, $t = 320$	margin $2^{-320}/(L + 2)$	certified exactly; 321 certificate bits
Typed recursion, 21 000 states	all states viable	2.1 s; linear in states
Action menu, 512 actions	all admissible, five modes	2.5 s
Beliefs, $m = 10$ labels	2^m reachable beliefs	1 024 enumerated, 0.59 s; bound fires at $m = 13$

Table 2: Scaling of the exact pipeline (full sweep; single core of a laptop CPU; the committed run is reproduced by `benchmarks/scaling_study.py`). Every instance’s answer is known by construction and asserted.

Artifact	Result
Benchmark exact checks (<code>safetransition verify</code>)	24/24 pass
Unit and integration test suite	46/46 pass
Runtime of checks + tests (laptop CPU)	under 4 s
Farkas certificate, stacked menu window	$\lambda = (1/2, 1/2)$, margin 1/10, re-verified
Independent checker over emitted certificates	3/3 verified; tampered certificates rejected
Scaling study, FM chain $k = 256$	closed-form margin verified; checker 0.29 s
Dashboard provenance	deterministic render; certificate SHA-256 embedded
Figure provenance	regenerated values equal deposited values (tested)
Adversarial exact/float instance ($n = 49$)	exact licensed; float would unlicense
Belief recursion, recourse-failure system	root belief in $\mathcal{W}_1 \setminus \mathcal{W}_2$
Source / test lines (plus checker, study)	1 633 / 621 (+250, +291)

Table 3: Verification artifacts shipped with SafeTransition 1.2.1. Every check and certificate is evaluated in exact rational arithmetic.

verification values: the witness datum and tube tables; quota admissibility on both disturbance branches ($0 \leq H^* \leq 13$, peak 1463/125); tube-status certification — the declared piecewise-linear plan tubes are exact by construction, while the Schaefer realization’s tubes are certified conservative outer enclosures, the enclosure following from monotonicity of σ on the visited biomass interval ($\sigma \geq 528/125 \geq 4$ on the adverse recovery leg); the index blindness of the composite reading at $w = (1, 1)$, whose minimum stays at $2/5 > 0$ along the licensed plan while the ecological floor reaches $-4/5 < 0$ mid-transition; the licensing thresholds $\rho_1 = 2/3$ and $\rho_2 = 3/2$, cross-checked against the thresholds derived directly from the tube geometry; and the rescue threshold $\kappa^* = 1 - x$ on the non-typed-viable region, truncated at zero ($\kappa^* = \max(0, 1 - x)$) once the fund covers the buy-back, with shortfall 1/2 at the false-positive witness and full financing at the rescue witness $x = 3/2$. All twenty-four checks pass in under a second on a laptop; Table 3 collects the verification artifacts.

Figure 1 shows the benchmark panels regenerated from the same verified values. Panel (a) is the separation a composite index cannot see: the weighted reading stays certified over the full review period while the floor it aggregates is driven below zero mid-transition. Panel (b) is the plan menu as management schedules: the pulse-plus-closed-season quota, the sustained-yield alternative at $\sigma(16/5) = 1088/125$, and the staged rebuild below yield while the fund finances the buy-back.

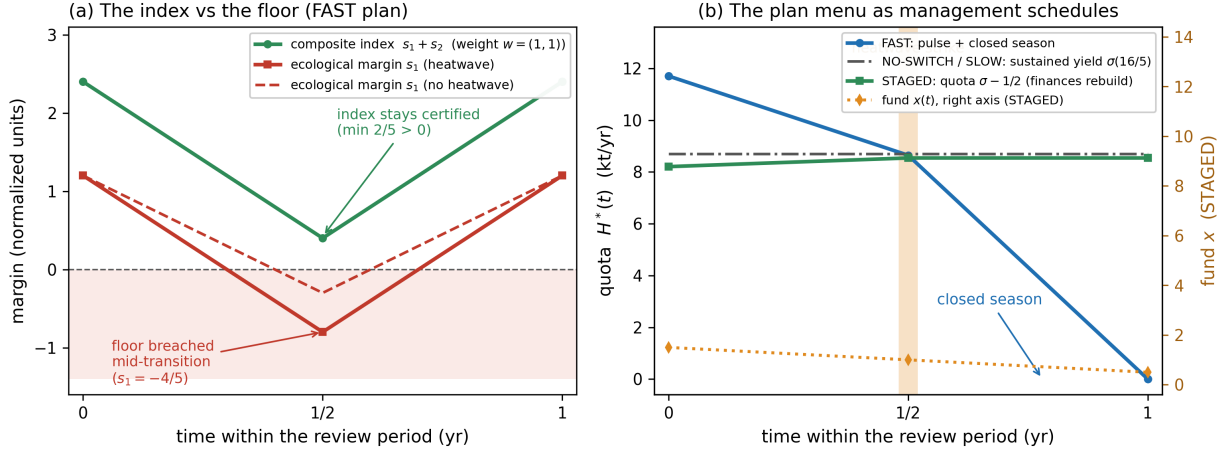


Figure 1: The resource-transition benchmark, regenerated from the library’s exact checks. (a) The composite index at $w = (1, 1)$ stays certified ($\min 2/5 > 0$) along the licensed plan while the ecological margin s_1 is breached mid-transition ($-4/5$ adverse, $-3/10$ benign): the separation the typed operator exposes. (b) The plan menu as management schedules: FAST (pulse 1463/125 plus closed season), SLOW/NO-SWITCH (sustained yield $\sigma(16/5)$), and STAGED (quota $\sigma - 1/2$, rebuild to 69/20), with the fund schedule $x(t)$ under STAGED on the right axis. Regenerated by `make_benchmark_v45.py` from the library’s verified schedule values, which equal the deposited values (equality tested).

4.2. Dashboard readings

The `compute` interface returns, at a state and weight, the per-plan floor minima under each plan’s characteristic disturbance, the composite index along the read plan, the licensing verdicts at the weight ratio $r = w_2/w_1$, the rescue threshold, and the index-blindness alarm, which fires exactly when the composite minimum stays positive while a floor minimum is negative. In this subsection the pairs $(1/2, 1/2)$ and $(3/2, 0)$ denote $(x, \kappa^*(x))$ — fund level and shortfall — not the phase-state coordinates (x, s_1, s_2) of Section 3.3. At the witness state $(x, s_1, s_2) = (1/2, 6/5, 6/5)$ with $w = (1, 1)$, the alarm fires: the readings reproduce the benchmark separation as a dashboard statement (Fig. 2). The `dashboard` module renders the readings, the benchmark status, and any certificates as a single HTML file with inline SVG and CSS and no external resources, so the file can be attached to a review, emailed, or archived with a statutory record and remains readable offline. The dashboard is an audit-delivery mechanism rather than a novelty claim, and it is bound to the certificate schema: a verification-provenance table embeds the library version, the independent checker’s version and re-run command, the datum identifier, and the SHA-256 hash of every certificate’s exact serialized JSON, expandable to the serialization itself. Rendering is deterministic — a pure function of the inputs, with no timestamps — so a re-render from the same inputs is byte-identical and the embedded hashes are checkable against the emitted certificate files. From the reviewer’s perspective this means the reading and its verification arrive together: thresholds, alarms, certificates, hashes, and the re-run command in one file.

4.3. Partial observation and certificates

The bundled demonstrations exercise the certificate family on the companion manuscript’s instances. The three-state recourse failure runs the belief-space recursion on a system whose root belief is one-step viable and exits at the second step under every observation-based policy: the root

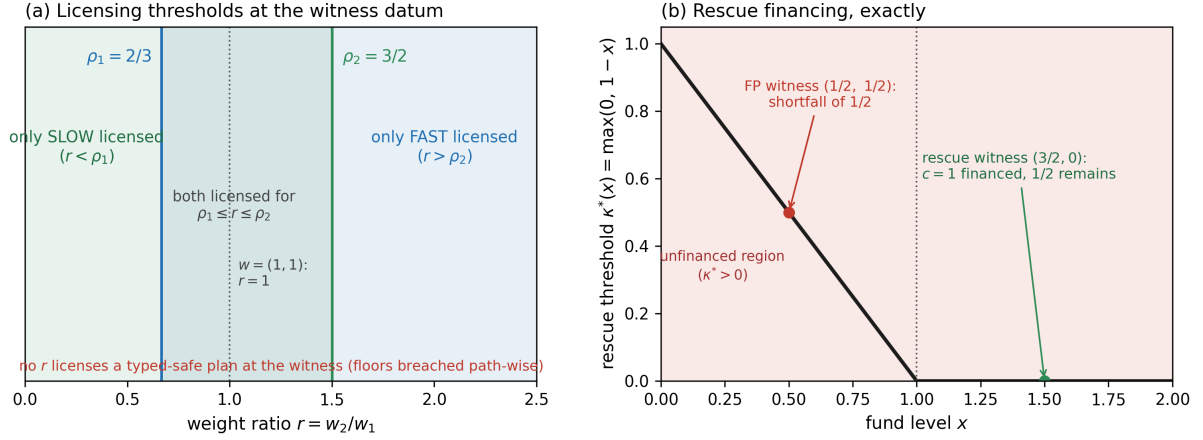


Figure 2: Dashboard readings at the witness datum, exact. (a) Per-weight licensing: the aggregate operator licenses FAST at weight ratios $r \geq \rho_1 = 2/3$ and SLOW at $r \leq \rho_2 = 3/2$; the standard equal-weight reading $r = 1$ sits inside the band where both are licensed and no ratio licenses a typed-safe plan. (b) Rescue financing: $\kappa^*(x) = \max(0, 1 - x)$; the false-positive witness $(1/2, 1/2)$ carries a shortfall of $1/2$, the rescue witness $(3/2, 0)$ finances the buy-back with $1/2$ remaining.

belief lies in $\mathcal{W}_1 \setminus \mathcal{W}_2$, witnessed only by the recursion. The observation-fibre criterion reports the merged fibre on which safe-set membership varies, ruling out any exact observation-only certifier. The stacked menu window $u \leq 2/5$, $-u \leq -3/5$ returns the Farkas certificate $\lambda = (1/2, 1/2)$ with infeasibility margin exactly $1/10$, re-verified against $\lambda^\top A = 0$ and $\lambda^\top b < 0$ before being returned. The mapping to the obstruction calculus is row-explicit: row $[1]$ $u \leq 2/5$ is the first state's menu cap, row $[-1]$ $u \leq -3/5$ is the second state's floor requirement, the certificate's multiplier vector weights the states' restrictions equally, and the empty common menu the obstruction reports is exactly the infeasibility these two rows certify — one Farkas object, addressable row by row to the states whose compatibility it denies. The pooled-kernel identity of the deposited double-integrator instance, weights $(3/8, 5/16, 5/16)$ with $\sum_j \lambda_j n_j = 0$, is checked exactly. On the recourse-failure system, `explain_belief_failure` returns the counterexample object: at the root belief both actions are admissible in isolation, and each fails because its post-belief is not one-step viable — the machine-checkable witness behind the $\mathcal{W}_1 \setminus \mathcal{W}_2$ verdict.

The weight-space analysis is complete rather than pointwise. The `weight_partition` interface returns the full arrangement of licensed plan sets along the ratio $r = w_2/w_1$ as a serializable certificate. For the benchmark floors the arrangement is the three regions the readings display: SLOW-only on $[0, 2/3)$, both plans licensed on $[2/3, 3/2]$, FAST-only on $(3/2, \infty)$. Across the rational family of floor pairs (s_1, s_2) and dip depths the arrangement has three regimes: for $\text{dip} < s_1 + s_2$ (the benchmark's) the FAST threshold $(\text{dip} - s_1)/s_2$ precedes the SLOW threshold $s_1/(\text{dip} - s_2)$; for $\text{dip} > s_1 + s_2$ the thresholds swap and an unlicensed gap opens around $r = 1$; at equality the middle region degenerates to the single point where both plans bind simultaneously. The independent checker re-derives, from the certificate alone, the thresholds, region coverage and inclusivity, the licensed set at a sampled ratio in each region, and the binding witness constraint at each boundary.

5. Relation to existing software

SafeTransition complements three established families of tools, and borrows from each. Numerical viability kernels and reachability tools (for the viability-theoretic tradition, and for general-purpose simulation frameworks in the environmental modelling literature, e.g. Raven [8] and FloPy [6]) compute set-valued safety objects to high floating-point precision with impressive scalability; they answer *how large is the safe set*, where SafeTransition answers *is this verdict verifiable*, for the small, structured, rational instances that assessment menus produce. Multi-criteria and sensitivity analysis packages — the SAFE toolbox [13], SALib [10], SPOTPY [11] — share the library’s workflow-oriented design: model-independent interfaces, worked examples, and benchmark functions; SPOTPY’s demonstration that one package can carry an entire family of methods with a minimal amount of glue code is the presentation standard SafeTransition aims at. Modular environmental simulators with public source code and regression suites — Pywr [14] and airGR [7] — inform the availability discipline: permissive licensing, distribution through package indexes, and a test suite executed on continuous integration. None of the environmental families returns certificates or computes in exact arithmetic. General-purpose provers do both in their own domains: the z3 satisfiability-modulo-theories solver [4] decides linear real arithmetic exactly over the rationals and produces checkable unsatisfiability cores, and the Fourier–Motzkin projection library pyfme [3] performs exact symbolic elimination over SymPy rationals with Imbert acceleration; probabilistic model checking in PRISM [5] verifies finite-state probabilistic models against temporal-logic specifications and returns counterexample traces at floating-point scale. What none of them ships is assessment semantics: floors, weight-licensed plan menus, reachability tubes for transition data, and epistemic viability sets are objects the analyst must encode afresh, and the encoding, not the prover, then carries the burden of exactness. Table 4 summarizes the comparison; the twelve verbatim search queries behind it (q1–q12, with dates, result records, and the mapping to the claims they support) are archived in the verification deposit [9]. The comparison is functional, not competitive: the families operate at scales and purposes for which floating-point computation is the right engineering choice, and the exact layer becomes decisive precisely where verdicts enter statutory or adversarial settings and must be re-derivable by third parties.

6. Availability and requirements

The test suite (46 tests) covers the benchmark, the operator chain and witnessed separation on the witness datum, the certificate family on the deposited instances, the certificate protocol (serialization, independent checking, tamper rejection, import hygiene, the adversarial exact/float instance), the scaling study’s closed-form answers on small instances, figure provenance (regenerated values equal deposited values), dashboard provenance determinism, the belief-space recursion on the recourse-failure system, and the dashboard’s self-containment; it runs in under four seconds. The dashboard example regenerates the single-file HTML from the library. Section-level derivations, the complete enumeration of the twenty-four benchmark checks, a worked certificate derivation, the package artifact tree with its reproduction procedure, and the related-software search records supporting Section 5 are provided in the Supplementary Material (S1–S8).

7. Conclusions

SafeTransition turns the assessment-operator framework and the obstruction calculus of the companion manuscripts into auditable software: typed operators evaluated on exact tubes, re-

Property	SafeTransition	z3	PRISM	pyfme
Exact arithmetic	rational throughout	yes (linear arithmetic)	no (floating point)	yes (SymPy rationals)
Verdicts by third parties	checkable yes; Farkas certificates, independent checker	unsat proofs	cores, counterexample traces	no provenance
Assessment tube semantics (floors, licensing)	built in	re-encoding required	re-probabilistic models	polyhedra only
Weighted-aggregation analysis	licensing per ratio; complete weight partition	encoding-level	—	—
Belief-space recursion	viability epistemic sets $\mathcal{W}_k$ with failure witnesses	encoding-level	POMDP algorithms (numerical)	—

Table 4: Functional comparison. The environmental toolsets discussed in the text (Raven, FloPy, Pywr, airGR, SAFE, SALib, SPOTPY) are numerical throughout and return no certificates; they are omitted from the columns for space.

cursions over state and belief space, and infeasibility returned as Farkas certificates with exact margins, serialized and re-verified by an independent checker sharing no code with the library. The library’s scope is deliberate: it certifies verdicts on small, explicit, rational instances under exact or certified conservative tubes, and it makes no predictive claim — the theorem-level guarantees are those of the companion manuscripts, and on continuous or belief-continuous systems the graph construction itself is the modelling step. Within that scope, the tool addresses a practical gap: readings of the kind composite dashboards display, and nonviability verdicts of the kind that justify tightening a plan menu, can carry their certificates with them wherever verdicts must be re-derivable by third parties. With independent checking, the scaling evidence, and the comparison of Section 5 in place, the tool closes this practical gap within its stated scope. Extensions under way are continuous-state data with certified enclosures, the extension of the weight partition to richer plan menus, and additional certificate families from the obstruction calculus; the interface intentionally leaves room for all three.

CRediT authorship contribution statement

A.A.: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of generative AI in the research process

Deepseek AI and Qwen (Alibaba) assisted with code development. The author reviewed and edited the content as needed and takes responsibility for the content.

Acknowledgements

Not applicable.

Name	SafeTransition
Developer and contact	Amin Abaee; amin_abaee@ut.ac.ir
Year first available	2026
Hardware required	none beyond a standard computer
Software required	Python ≥ 3.9 ; no packages beyond the standard library (matplotlib and Pillow only for regenerating figures; publicly maintained third-party distributions are listed with their pinned versions in the verification deposit)
Program language	Python
Program size	1633 source lines, 621 test lines, 250-line independent checker, 291-line scaling study
Availability and cost	open source under the MIT licence; source, tests, and examples archived in the verification deposit [9] and mirrored in the project repository; no cost

Table 5: Software availability summary (EMS required fields).

References

- [1] Abaee, A., 2026a. Aggregate Indices and Transition Safety: A Quantifier-Order Separation Between Scalarized and Coordinate-Wise Feasibility. Manuscript submitted for publication.
- [2] Abaee, A., 2026b. An Obstruction Calculus for Viability under Incomplete Observation. Manuscript submitted for publication.
- [3] Caron, S., 2022. pyfme: symbolic polyhedron projection by Fourier–Motzkin elimination using SymPy. Python package, version of 2 December 2022. Available at: <https://github.com/stephane-caron/pyfme>.
- [4] de Moura, L., Bjørner, N., 2008. Z3: an efficient SMT solver, in: Ramakrishnan, C.R., Rehof, J. (Eds.), Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Budapest, Hungary, March 29–April 6, 2008. Proceedings. Springer, Berlin, Heidelberg, pp. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24.
- [5] Kwiatkowska, M., Norman, G., Parker, D., 2011. PRISM 4.0: verification of probabilistic real-time systems, in: Gopalakrishnan, G., Qadeer, S. (Eds.), Computer Aided Verification, 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14–20, 2011. Proceedings. Springer, Berlin, Heidelberg, pp. 585–591. https://doi.org/10.1007/978-3-642-22110-1_47.
- [6] Bakker, M., Post, V., Langevin, C.D., Hughes, J.D., White, J.T., Starn, J.J., Fienen, M.N., 2016. Scripting MODFLOW model development using open source Python and FloPy. Groundwater 54 (5), 733–739. <https://doi.org/10.1111/gwat.12413>.
- [7] Coron, L., Thirel, G., Delaigue, O., Perrin, C., Andréassian, V., 2017. The suite of lumped GR hydrological models in an R package. Environmental Modelling & Software 94, 166–171. <https://doi.org/10.1016/j.envsoft.2017.05.002>.
- [8] Craig, J.R., Brown, G., Chlumsky, R., Jenkinson, R.W., Jost, G., Lee, K., Mai, J., Serrer, M., Sgro, N., Shafii, M., Snowdon, A.P., Tolson, B.A., 2020. Flexible watershed simulation with the Raven hydrological modelling framework. Environmental Modelling & Software 129, 104728. <https://doi.org/10.1016/j.envsoft.2020.104728>.
- [9] Abaee, A., 2026. Verification code and figure pipeline for Aggregate Indices and Transition Safety [data set]. figshare. <https://doi.org/10.6084/m9.figshare.33764023>.
- [10] Herman, J., Usher, W., 2017. SALib: An open-source Python library for sensitivity analysis. Journal of Open Source Software 2 (9), 97. <https://doi.org/10.21105/joss.00097>.
- [11] Houska, T., Kraft, P., Chamorro-Chavez, A., Breuer, L., 2015. SPOTting model parameters using a ready-made Python package. PLOS ONE 10 (12), e0145180. <https://doi.org/10.1371/journal.pone.0145180>.
- [12] Jakeman, A.J., Letcher, R.A., Norton, J.P., 2006. Ten iterative steps in development and evaluation of environmental models. Environmental Modelling & Software 21 (5), 602–614. <https://doi.org/10.1016/j.envsoft.2006.01.004>.
- [13] Pianosi, F., Sarrazin, F., Wagener, T., 2015. A Matlab toolbox for global sensitivity analysis. Environmental Modelling & Software 70, 80–85. <https://doi.org/10.1016/j.envsoft.2015.04.009>.
- [14] Tomlinson, J.E., Arnott, J.H., Harou, J.J., 2020. A water resource simulator in Python. Environmental Modelling & Software 126, 104635. <https://doi.org/10.1016/j.envsoft.2020.104635>.